RHEO — Complete Systems & Cloud Engineering Interview Handbook

Real-Time Relay File Transfer System

Author: Ganesh Daware | Stack: Node.js, Express, React, Socket.IO, PostgreSQL (Prisma), Redis, Docker, AWS EC2, Nginx, CI/CD

---

1. Project Overview & Pitch

"Tell me about RHEO"

> *"RHEO is a production-deployed, cloud-native real-time file sharing platform designed for multi-user, chunked relay transfers of files up to 5 GB. Instead of relying on complex WebRTC mesh topologies that frequently fail behind corporate firewalls and symmetric NATs, RHEO uses an application-level relay architecture built on persistent WebSockets (Socket.IO), Redis for presence and pub/sub, and PostgreSQL for durable state. It features flow control (sliding window of 8 in-flight chunks), resumable transfers via periodic database checkpoints, multi-recipient fan-out to up to 50 concurrent receivers, and ephemeral collaborative rooms with real-time chat."*

What problem does RHEO solve?

1. WebRTC P2P Fragility: WebRTC peer-to-peer transfers frequently drop or fail to establish connections due to Symmetric NATs, firewalls, and carrier-grade NATs (CGNAT), requiring expensive TURN relay infrastructure.

2. Cloud Storage Overhead: Traditional cloud drive services (Dropbox/Google Drive) force full uploads to S3/blob storage before the recipient can begin downloading, creating high latency and massive server storage costs.

3. RAM Bloat on Large Transfers: Uploading multi-gigabyte files via standard HTTP multipart forms buffers full files or large streams in memory, causing server memory exhaustion (OOM).

4. Network Dropouts: Standard browser uploads fail completely if Wi-Fi blips for 2 seconds. RHEO checkpoints progress every 50 chunks, allowing seamless transfer resume.

What does RHEO stand for?

RHEO is derived from the Greek word *rheo* (■■ω), meaning **"to flow"** or **"stream"** — symbolizing continuous, low-latency, backpressure-controlled data streaming across nodes.

Main Features:

• **1 MB Chunked WebSocket Streaming**: Relays binary data in real-time with zero full-file buffering on the server.

• **Application Flow Control**: Enforces `MAX_IN_FLIGHT_CHUNKS = 8` to protect client & server RAM against fast senders and slow receivers.

• **Durable Checkpoint Resumes**: Automatic resume from last ACK'd chunk even after browser reloads, socket drops, or EC2 instance replacements.

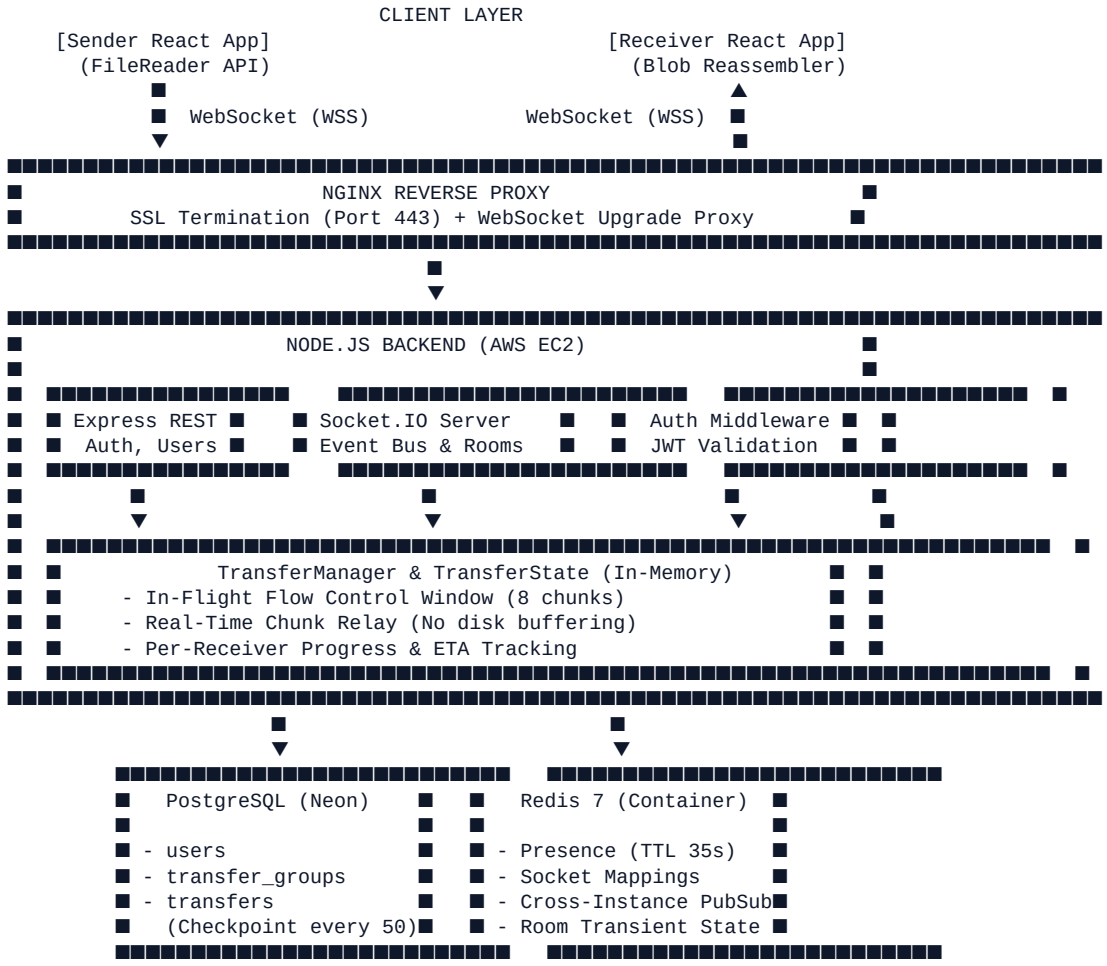
• **1-to-N Multi-Recipient Fan-Out**: Single upload fans out to up to 50 independent receiver streams with isolated flow control.

• **Redis Presence & Rooms**: 35-second TTL heartbeats and in-memory collaborative rooms with 6-digit join codes and real-time chat.

• **End-to-End Integrity**: Client-side SHA-256 checksum computed on reassembly.

• **Automated CI/CD**: Zero-downtime automated deployment to AWS EC2 (backend) and Vercel (frontend) via GitHub Actions.

Architecture Diagram:



---

2. Deep Dive: WebSockets & Socket.IO

Why WebSockets instead of HTTP/REST for file transfer?

- **Bidirectional, Full-Duplex Connection**: HTTP requires request-response polling. WebSockets open a single persistent TCP connection where both sender and receiver push data with ~2 bytes of frame overhead (compared to 800+ bytes of HTTP headers per request).
- **Sub-Millisecond Real-Time Relay**: When a sender pushes chunk 45, the server immediately forwards it down the receiver's open socket without waiting for the receiver to poll `/api/get-next-chunk`.
- **Lower Latency & CPU Overhead**: Eliminates repeated TLS handshakes, TCP connection setup, and HTTP header parsing for thousands of chunks.

How does the WebSocket Handshake work?

1. Client sends a standard HTTP GET request with upgrade headers:

```
GET /socket.io/?EIO=4&transport=websocket HTTP/1.1
Host: api.rheo.app
Upgrade: websocket
Connection: Upgrade
Sec-WebSocket-Key: dGhlIHNhbXBsZSBub25jZQ==
Sec-WebSocket-Version: 13
```

2. Nginx forwards the `Upgrade` and `Connection` headers to Node.js.
3. Node.js/Socket.IO responds with HTTP Status `101 Switching Protocols`:

```
HTTP/1.1 101 Switching Protocols
Upgrade: websocket
Connection: Upgrade
Sec-WebSocket-Accept: s3pPLMBiTxaQ9kYGzzhZRbK+xOo=
```

4. The TCP socket remains open, switching from HTTP text frames to WebSocket binary/text frames.

Is WebSocket TCP or UDP? Does it provide reliability and ordering?

• WebSocket runs on top of **TCP** (Transmission Control Protocol).

• **TCP guarantees**: Byte-level reliable delivery, packet retransmission on packet loss, in-order byte stream delivery, and network-level congestion control.

Follow-Up Trap: "If TCP guarantees reliability, why did you implement Application-Level ACKs?"

> ***Critical Interview Answer**:*
>
> *"TCP guarantees byte delivery between the two network interfaces (NICs). However, TCP ACK only means the operating system's kernel buffer received the TCP segment. It does **not** guarantee that:*
>
> *1. The receiver's JavaScript engine parsed the binary payload.*
>
> *2. The client successfully wrote the chunk to IndexedDB/memory without crashing.*
>
> *3. The transfer state was preserved across an abrupt browser refresh or network switch.*

>

> *Our application-level* `CHUNK_ACK` *represents **application processing confirmation**, drives the sliding window backpressure flow control, and provides checkpoints in PostgreSQL for resume-after-disconnection."*

Socket.IO vs Native WebSockets:

• **Automatic Reconnection**: Socket.IO has built-in exponential backoff reconnection.

• **Heartbeat / Ping-Pong**: Detects half-open "zombie" connections (`pingInterval: 25000`, `pingTimeout: 60000`).

• **Room Multiplexing**: Native support for logical channels (`socket.join('room:XYZ')`, `socket.to('user:UUID').emit()`).

• **Binary & JSON Serialization**: Handles ArrayBuffers seamlessly alongside structured JSON metadata.

---

3. File Chunking & Streaming Mechanics

Why chunk files?

1. Memory Safety: Loading a 5 GB file into browser or server RAM causes heap crashes (V8 has a default ~1.4–4 GB heap limit).

2. Resilience: If a transfer drops at 4.8 GB, you only retransmit the remaining unconfirmed 1 MB chunks, not the entire 5 GB.

3. Pipelining & Progress: Enables streaming progress bars, instant ETA calculations, and application flow control.

Chunk Size Math (5 GB File):

• **Selected Chunk Size**: `1 MB` (`1,048,576 bytes`).

• **5 GB in Bytes**: `5 *1024* 1024 * 1024 = 5,368,709,120 bytes`.

• **Total Chunks Calculation**:

$$\text{Total Chunks} = \lceil \frac{5,368,709,120}{1,048,576} \rceil = 5,120 \text{ chunks}$$

• **Last Chunk**: Exactly $5,368,709,120 \pmod{1,048,576} = 0$ (or the remainder bytes if not an exact multiple). The slice is `file.slice(5119 * 1048576, file.size)`.

Why 1 MB chunk size?

• **Smaller Chunks (e.g., 64 KB)**: High frame overhead (5,120 * 16 = 81,920 socket messages), higher CPU interrupt overhead, and higher latency due to excessive ACK processing.

• **Larger Chunks (e.g., 20 MB)**: High retransmission cost on failed chunks, coarse progress updates, and potential WebSocket buffer saturation.

• **1 MB is the sweet spot**: Optimal balance between frame throughput, minimal JS event loop overhead, and responsive progress feedback.

Chunk Payload Structure:

```
{
  transferId: "a1b2c3d4-...",  // UUID identifying transfer
  chunkIndex: 42,              // 0-indexed sequence number
  totalChunks: 5120,           // Total expected
  chunkData: ArrayBuffer(1MB)  // Raw binary payload
}
```

Complete End-to-End Walkthrough (Clicking Send to Completion):

1. Selection & Hashing: Sender selects `archive.iso` (5 GB). Browser reads file in background chunks via Web Workers using `SubtleCrypto` to compute SHA-256 hash.

2. Handshake Request: Sender emits `TRANSFER_REQUEST` to server with `{ receiverIds, fileName, fileSize, totalChunks, fileHash }`.

3. Database Staging: Server validates `fileSize <= 5GB`, generates a `TransferGroup` in PostgreSQL, stages `Transfer` records with status `PENDING`, and notifies receiver sockets via `TRANSFER_REQUEST`.

4. Receiver Acceptance: Receiver clicks Accept $\rightarrow$ emits `TRANSFER_ACCEPTED`. Server updates status to `TRANSFERRING` and creates in-memory `TransferState`.

5. Pipelined Streaming:

• Sender reads Chunk 0 via `file.slice(0, 1MB)` and emits `CHUNK`.

• Server relays Chunk 0 to receiver's socket room (`user:<receiverId>`).

• Server updates in-memory `inFlightChunks++` and `nextChunkToSend = 1`.

• Sender sends up to `MAX_IN_FLIGHT_CHUNKS` (8 chunks) without waiting.

6. ACK & Flow Control Sliding Window:

• Receiver receives Chunk 0, stores ArrayBuffer in client memory array/IndexedDB, and emits `CHUNK_ACK` with `{ transferId, chunkIndex: 0 }`.

• Server decrements `inFlightChunks--`, updates `lastConfirmedChunk = 0`, and forwards ACK to sender.

• Sender slides window and emits Chunk 8.

7. Periodic Checkpoints: Every 50 chunks (`DB_CHECKPOINT_INTERVAL`), the server writes `lastConfirmedChunk` asynchronously to PostgreSQL.

8. Reassembly & Hash Verification:

• Upon receiving Chunk 5119, receiver concatenates chunks into a `Blob`, emits final `CHUNK_ACK`.

• Server marks PostgreSQL transfer `COMPLETED`, emits `HASH_VERIFY` with expected SHA-256.

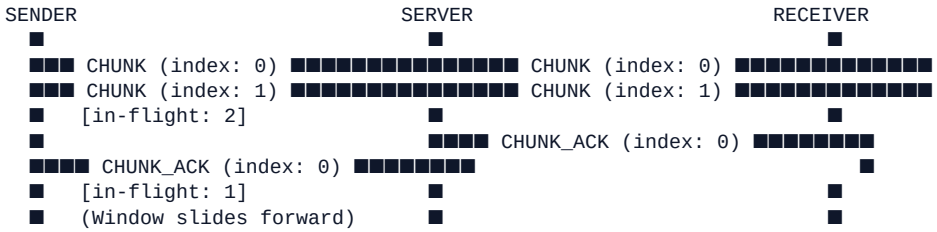
• Receiver calculates SHA-256 over reconstructed Blob; emits `HASH_RESULT` with `{ verified: true }`.

• Browser triggers automatic download via `URL.createObjectURL(blob)`.

---

4. Application-Level Acknowledgements (ACKs) & Backpressure

ACK Mechanism:



What happens if an ACK is lost?

Because Socket.IO runs over TCP, the physical packet is not dropped under normal conditions. However, if a socket connection terminates abruptly before an ACK arrives:

1. The sender's in-flight window stops advancing.

2. After reconnection, the sender queries the server via `TRANSFER_RESUME_REQUEST`.

3. The server checks the last durable checkpoint in PostgreSQL (`lastConfirmedChunk = 49`) and instructs the sender: *"Resume from chunk 50"*.

4. Any duplicate chunks received are idempotent because the receiver indexes chunks into an array: `chunks[chunkIndex] = chunkData`.

---

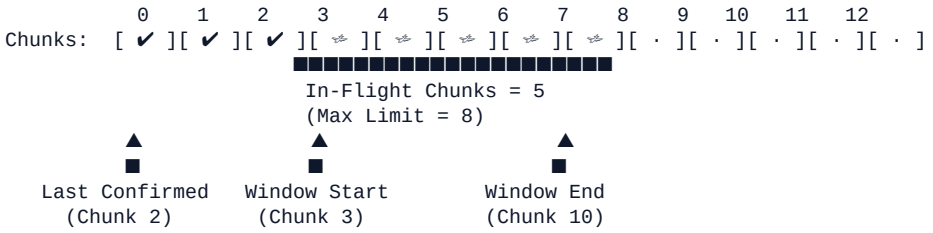
5. Flow Control & Sliding Window

What is Flow Control vs Congestion Control?

• **Congestion Control (TCP layer)**: Prevents the network between sender and server from being overloaded with packets.

• **Flow Control (Application layer)**: Prevents a fast sender from overwhelming a slow receiver's CPU/RAM or the server's socket buffer.

Whiteboard Diagram — Application Sliding Window:



• When `inFlightChunks == MAX_IN_FLIGHT_CHUNKS` (8), the sender's client pauses file slicing.

• As `CHUNK_ACK` events arrive, `inFlightChunks` decrements, allowing the next chunks to be read and transmitted.

---

6. Connection Recovery & Resume Mechanics

Scenario Walkthrough: 5 GB transfer at 3.2 GB (~Chunk 3200) Wi-Fi disconnects for 10 minutes:

1. Disconnection Event: Client loses Wi-Fi. Socket.IO detects TCP drop (`disconnect` event fires).

2. Server State Capture: Server `handleDisconnect` marks the transfer status as `INTERRUPTED` in PostgreSQL and saves `lastConfirmedChunk = 3200`. In-memory `TransferState` is cleared or marked interrupted.

3. Client Reconnection: Wi-Fi reconnects after 10 minutes. Socket.IO automatically reconnects and re-authenticates with JWT.

4. Resume Request: Sender emits `TRANSFER_RESUME_REQUEST` with `{ transferId }`.

5. State Reconstruction:

• Server executes `SELECT * FROM transfers WHERE id = transferId`.

• Validates user authorization (`senderId === userId || receiverId === userId`).

• Reconstructs in-memory `TransferState` with `lastConfirmedChunk = 3200`.

• Updates PostgreSQL status back to `TRANSFERRING`.

6. Continuation: Server replies with `resumeFromChunk: 3201`. Sender's browser seeks file to byte offset `3201 * 1,048,576` and resumes streaming from chunk 3201.

7. Result: Zero data lost; 3.2 GB does not need to be retransmitted.

---

7. Multi-Recipient Transfer (1-to-N Fan-Out)

How does 1-to-N Multi-Recipient work?

• Sender specifies an array of `receiverIds: ["UUID-1", "UUID-2", "UUID-3"]`.

• The database creates **1** `TransferGroup` (metadata: file name, size, hash) and **N distinct** `Transfer` **records**.

• Each recipient runs an **independent** `TransferState` in server memory:

• Receiver 1 on 100 Mbps fiber $\rightarrow$ at Chunk 4500.

• Receiver 2 on 4G mobile $\rightarrow$ at Chunk 1200.

• **Isolated Backpressure**: Slower receiver B never blocks or slows down faster receiver A because ACKs and flow control windows are tracked on separate `TransferState` instances.

Scaling Multi-Recipient to 10,000 Receivers (System Design):

If 10,000 users request a file simultaneously:

1. Hybrid Storage S3 Offloading: Instead of relaying 10,000 WebSocket streams from one browser (which saturates client upload bandwidth), the sender uploads **once** to AWS S3 via multipart presigned URLs.

2. CDN Distribution: Receivers download chunks via AWS CloudFront CDN edge locations.

3. WebSocket for Control Plane Only: Socket.IO handles signaling, room coordination, chat, and download start triggers, while the heavy binary data plane shifts to edge CDN caching.

---

8. Redis Architecture & Presence

Why Redis?

• **Sub-millisecond latency**: In-memory key-value operations for high-frequency presence checks.

• **Atomic TTL Expiration**: Eliminates zombie "online" users when laptops sleep without clean socket disconnects.

• **Pub/Sub for Horizontal Scaling**: Broadcasts socket events across multiple Node.js EC2 instances.

Redis Keys & Data Structures:

| Key Pattern | Data Structure | TTL | Purpose |

|---|---|---|---|

| `presence:user:<userId>` | String (`"online"`) | 35s | User online indicator |

| `socket:user:<userId>` | String (`socketId`) | 35s | Maps user ID to active socket |

| `socket:id:<socketId>` | String (`userId`) | 35s | Reverse lookup on socket disconnect |

| `network:peers:<groupId>` | Set | 35s | Local network discovery peers |

How Heartbeat Presence Works:

1. When connected, server sets `SET presence:user:<userId> "online" EX 35`.

2. Client sends a ping every 25 seconds; server refreshes the 35s TTL.

3. If user loses power/Wi-Fi without a `disconnect` packet, Redis automatically expires the key after 35 seconds, preventing stale presence.

---

9. PostgreSQL & Database Design

Schema Design (Prisma ORM):

- `users`: `id (UUID PK)`, `username (Unique)`, `email (Unique)`, `password_hash`, `avatar_id`, `created_at`, `last_seen`.
- `transfer_groups`: `id (UUID PK)`, `sender_id (FK)`, `file_name`, `file_size (BigInt)`, `total_chunks (Int)`, `file_hash (VarChar 64)`.
- `transfers`: `id (UUID PK)`, `group_id (FK)`, `sender_id (FK)`, `receiver_id (FK)`, `status (Enum)`, `last_confirmed_chunk (Int)`, `total_chunks (Int)`.

Indexing Strategy:

- `CREATE INDEX idx_transfers_sender ON transfers(sender_id);` — Fast retrieval of sent files.
- `CREATE INDEX idx_transfers_receiver ON transfers(receiver_id);` — Fast retrieval of incoming files.
- `CREATE INDEX idx_transfers_status ON transfers(status);` — Instant filtering for active/interrupted transfers.
- `CREATE INDEX idx_transfers_group ON transfers(group_id);` — Joins between transfer group metadata and recipients.

---

10. Security & Authentication (JWT + WebSockets)

JWT Token Structure:

- **Header**: Algorithm & token type (`{"alg": "HS256", "typ": "JWT"}`).
- **Payload**: User claims (`{"userId": "UUID", "email": "...", "iat": 1700000000, "exp": 1700604800}`).
- **Signature**: `HMACSHA256(base64UrlEncode(header) + "." + base64UrlEncode(payload), JWT_SECRET)`.
- *Note: JWT is cryptographically signed (tamper-proof), not encrypted.*

WebSocket Authentication:

Authentication occurs during the initial handshake before socket connection approval:

```
// server/src/sockets/index.js
io.use(async (socket, next) => {
  try {
    const token = socket.handshake.auth?.token;
    if (!token) return next(new Error('Authentication required'));

    const decoded = verifyToken(token); // Verifies signature & expiration
    socket.userId = decoded.userId;     // Authoritative identity derivation
    next();
  } catch (err) {
    next(new Error('Invalid or expired token'));
  }
});
```

---

11. Docker & Containerization

Multi-Stage Dockerfile (`server/Dockerfile`):

```
# Stage 1: Build & Dependencies
FROM node:20-alpine AS builder
WORKDIR /build
COPY package*.json ./
COPY prisma ./prisma/
RUN npm ci
RUN npx prisma generate

# Stage 2: Minimal Production Runtime
FROM node:20-alpine AS runtime
WORKDIR /app
USER node
COPY --from=builder --chown=node:node /build/node_modules ./node_modules
COPY --from=builder --chown=node:node /build/prisma ./prisma
COPY --chown=node:node src/ ./src/
COPY --chown=node:node package.json ./

EXPOSE 5000
HEALTHCHECK --interval=30s --timeout=10s --retries=3 \
  CMD wget -qO- http://localhost:5000/health || exit 1

CMD ["sh", "-c", "npx prisma migrate deploy || true; node src/server.js"]
```

Key Production Docker Optimizations:

1. **Multi-Stage Build**: Keeps compiler tools and dev dependencies out of the final runtime image, reducing image size by ~60%.
2. **Security (`USER node`)**: Runs process as unprivileged user instead of root.
3. **Database Migration on Startup**: Automatically executes `prisma migrate deploy` before launching HTTP server.

---

12. AWS EC2, Nginx & Production Infrastructure

Traffic Flow:

```
Client Browser (HTTPS / WSS)
             ■  Port 443
             ▼
      [AWS Security Group] (Allows 80, 443, 22)
             ■
             ▼
       [Nginx Reverse Proxy] (SSL Termination via Let's Encrypt)
             ■
   ■■■■■■■■■■■■■■■■■■■■■■■
   ■ (HTTP Proxy)      ■ (WebSocket Upgrade Proxy)
   ▼                   ▼
 Express REST API    Socket.IO Server
   (Port 5000)         (Port 5000)
```

Nginx WebSocket Upgrade Configuration (`nginx/nginx.prod.conf`):

```
location /socket.io/ {
    proxy_pass http://rheo_server:5000;
    proxy_http_version 1.1;
    proxy_set_header Upgrade $http_upgrade;
    proxy_set_header Connection "Upgrade";
    proxy_set_header Host $host;
    proxy_set_header X-Real-IP $remote_addr;
    proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
    proxy_read_timeout 86400s;
    proxy_send_timeout 86400s;
}
```

---

13. Comprehensive Failure Scenarios & Edge Cases

| Scenario | What Happens | System Handling |

|---|---|---|

| **Sender disconnects at 50%** | TCP socket drops | Server marks PostgreSQL status `INTERRUPTED`. On reconnect, server instructs sender to resume from `lastConfirmedChunk + 1`. |

| **Receiver disconnects** | Sender gets no more ACKs | Sender window fills (`inFlightChunks == 8`) and pauses slicing. Receiver reconnects and resumes without data retransmission. |

| **Server crashes mid-transfer** | In-memory `TransferState` lost | On restart, PostgreSQL checkpoints preserve progress. Reconnecting clients trigger `TRANSFER_RESUME_REQUEST` to rebuild state. |

| **Redis fails** | Presence checks fail | Socket relay continues functioning; presence gracefully falls back to degraded local state. |

| **PostgreSQL is slow** | DB writes delayed | Checkpointing runs asynchronously in the background (`transferService.checkpointTransfer().catch()`) without blocking the real-time chunk streaming path. |

| **Duplicate Chunk Arrives** | Network race condition | Receiver stores chunks in indexed array `chunks[chunkIndex] = data`, making duplicate chunks idempotent. |

| **Corrupted File on Reassembly** | Bit flips / dropped bytes | Receiver computes client-side SHA-256 hash. If hash does not match `fileHash`, marks transfer `FAILED` and prompts retry. |

| **Tab Closed / Reopened** | Socket ID changes | JWT auth re-identifies `userId`; active transfers for user are retrieved from database and resumed. |

---

14. "Show Me The Code" Repository Guide

When asked to open the repo and show code:

- **WebSocket Chunk Relay & Flow Control**: [`server/src/sockets/chunkHandler.js`](file:///c:/Users/Ganesh%20Daware/OneDrive/sharing%20app%20rheo/Rheo/server/src/sockets/chunkHandler.js#L55-L145)
- **In-Memory Flow Control Window**: [`server/src/transfer/TransferState.js`](file:///c:/Users/Ganesh%20Daware/OneDrive/sharing%20app%20rheo/Rheo/server/src/transfer/TransferState.js#L53-L89)
- **Checkpoint Resume Handler**: [`server/src/sockets/recoveryHandler.js`](file:///c:/Users/Ganesh%20Daware/OneDrive/sharing%20app%20rheo/Rheo/server/src/sockets/recoveryHandler.js#L36-L105)
- **Multi-Recipient Handshake**: [`server/src/sockets/transferHandler.js`](file:///c:/Users/Ganesh%20Daware/OneDrive/sharing%20app%20rheo/Rheo/server/src/sockets/transferHandler.js#L35-L95)
- **Redis Presence TTL Implementation**: [`server/src/redis/presence.js`](file:///c:/Users/Ganesh%20Daware/OneDrive/sharing%20app%20rheo/Rheo/server/src/redis/presence.js#L24-L80)
- **JWT Socket Auth Middleware**: [`server/src/sockets/index.js`](file:///c:/Users/Ganesh%20Daware/OneDrive/sharing%20app%20rheo/Rheo/server/src/sockets/index.js#L79-L98)
- **Database Schema**: [`server/prisma/schema.prisma`](file:///c:/Users/Ganesh%20Daware/OneDrive/sharing%20app%20rheo/Rheo/server/prisma/schema.prisma#L20-L105)
- **Production Multi-Stage Dockerfile**: [`server/Dockerfile`](file:///c:/Users/Ganesh%20Daware/OneDrive/sharing%20app%20rheo/Rheo/server/Dockerfile#L1-L32)
- **CI/CD Pipeline**: [`.github/workflows/deploy.yml`](file:///c:/Users/Ganesh%20Daware/OneDrive/sharing%20app%20rheo/Rheo/.github/workflows/deploy.yml#L1-L125)